MODULE 10 · BLOCK 2 - CONTROL FLOW - CONDITIONAL BLOCKS AND LOOPS

Loop patterns the exam loves

Accumulating, counting, finding the largest, searching, working with digits, and bubble sort.

BLOCK 2 of 4 EXAM WEIGHT 29% MODULE 10 of 28

admin@sthithapragna.com

Most loop questions are one of six patterns. **Check the starting value first**: it is where the traps are hidden.

Recognise the pattern, then check its start

PATTERN	STARTS WITH	EACH PASS
Accumulate	total = 0, or product = 1	total += n product *= n
Count	count = 0	if condition: count += 1
Largest or smallest	best = data[0]	if n > best: best = n
Search	found = False	if match: found = True; break
Digits	a whole number n	n % 10 is the last digit; n //= 10 drops it
Bubble sort	an unsorted list	swap neighbours that are out of order

ON THE EXAM

The starting value is the most common trap in every one of these patterns.

A sum starts at 0. A product starts at 1

```
1  nums = [2, 3, 4]
2  total = 0
3  product = 1
4  for n in nums:
5      total += n
6      product *= n
7  print(total, product)       # 9 24
```

2 0 is the neutral value for adding: 0 + n is n.

3 1 is the neutral value for multiplying. Start at 0 and the product is 0 for ever.

Add one only when the condition holds

```
1  words = ["apple", "kiwi", "banana", "fig"]
2  count = 0
3  for w in words:
4      if len(w) > 3:
5          count += 1
6  print(count)                # 3
```

4-5 count goes up by one, not by the value. apple, kiwi and banana qualify; fig does not.

Start with the first element, not with 0

STARTS AT 0

Wrong for all-negative data

```
1  data = [-5, -2, -9]
2  largest = 0
3  for v in data:
4      if v > largest:
5          largest = v
6  print(largest)
```

Prints 0. No value beats 0, so the starting value is never replaced.

STARTS AT DATA[0]

Always a real value

```
1  data = [-5, -2, -9]
2  largest = data[0]
3  for v in data:
4      if v > largest:
5          largest = v
6  print(largest)
```

Prints -2. The best so far is always a value from the list.

THE TRAP

If the answer is 0 and 0 is not in the data, suspect the starting value.

The same pattern with nested loops

```
1  grid = [[4, 9], [7, 2], [5, 8]]
2  smallest = grid[0][0]
3  for row in grid:
4      for value in row:
5          if value < smallest:
6              smallest = value
7  print(smallest)              # 2
```

2 grid[0][0] is the first number of the first row: 4.

3-6 The outer loop takes each row, the inner loop each number in it.

Python also has min(), max() and sum(). The exam uses both the loops and the built-ins.

Two ways to report whether something was found

A FLAG

Remember the result

```
found = False
for n in [4, 7, 1]:
    if n == 7:
        found = True
        break
print(found)
```

Prints True. The flag keeps the answer for later code.

FOR WITH ELSE

Report straight away

```
for n in [4, 7, 1]:
    if n == 7:
        print("found")
        break
else:
    print("not found")
```

Prints found. The else would run only if no break fired.

ON THE EXAM

Both patterns break on the first match, so later matches are never seen.

Testing for a prime with for and else

```
1  n = 13
2  for d in range(2, n):
3      if n % d == 0:
4          print(n, "is not prime")
5          break
6  else:
7      print(n, "is prime")        # 13 is prime
```

2-5 Try every possible divisor from 2 up to n - 1. Any exact division means not prime.

6-7 No break means no divisor was found.

Trap: for n = 1, range(2, 1) is empty, so the else runs and 1 is reported as prime.

% 10 takes the last digit; // 10 removes it

```
1  n = 472
2  total = 0
3  while n > 0:
4      total += n % 10          # 2, then 7, then 4
5      n //= 10                 # 47, then 4, then 0
6  print(total)                 # 13
```

4 472 % 10 is 2: the remainder after dividing by 10 is the last digit.

5 472 // 10 is 47: floor division by 10 chops the last digit off.

Build the reverse one digit at a time

```
1  n = 472
2  rev = 0
3  while n > 0:
4      rev = rev * 10 + n % 10    # 2, then 27, then 274
5      n //= 10
6  print(rev)                     # 274
```

4 Times 10 shifts the digits built so far one place left, making room for the new last digit.

Zeros on the end disappear: 120 reverses to 21, because the reverse would start with 0.

Bubble sort, from Python Essentials 1

```
1  data = [8, 10, 6, 2, 4]
2  swapped = True
3  while swapped:
4      swapped = False
5      for i in range(len(data) - 1):
6          if data[i] > data[i + 1]:
7              data[i], data[i + 1] = data[i + 1], data[i]
8              swapped = True
9  print(data)                    # [2, 4, 6, 8, 10]
```

5 len(data) - 1, because each step compares i with i + 1. Going one further would be an IndexError.

7 The swap from module two: both right-hand values are read before either slot changes.

3-4 Repeat whole passes until one pass makes no swaps at all.

Each pass carries the largest remaining value to the end

AFTER PASS	LIST	SWAPS MADE?
1	[8, 6, 2, 4, 10]	yes: 10 has bubbled to the end
2	[6, 2, 4, 8, 10]	yes: 8 is in place
3	[2, 4, 6, 8, 10]	yes: sorted, but the loop does not know yet
4	[2, 4, 6, 8, 10]	no: the loop ends

ON THE EXAM

The last pass changes nothing. It is needed to prove the list is sorted.

Functions that replace these loops

- **sum(nums)**, **min(nums)**, **max(nums)** and **len(nums)** do the accumulating and searching.
- **sorted(nums)** returns a new sorted list; the original is unchanged.
- **nums.sort()** sorts in place and returns **None**. print(nums.sort()) prints None.
- The exam uses the loops to test tracing, and the built-ins to test what they return.

What to carry into the exam

- Sums start at 0; **products start at 1**.
- Largest and smallest start at **the first element**, never 0.
- A search with break stops at the **first** match. for-else reports no match.
- n % 10 is the last digit; n // 10 removes it. 120 reversed is 21.
- Bubble sort compares neighbours, uses range(len(data) - 1), and stops after a pass with no swaps.

What is the output of this program?

```
1  nums = [3, 1, 4]
2  result = 0
3  for n in nums:
4      result = result * 10 + n
5  print(result)
```

A 314

B 8

C 413

D 3140

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  nums = [3, 1, 4]
2  result = 0
3  for n in nums:
4      result = result * 10 + n
5  print(result)
```

0 * 10 + 3 = 3. 3 * 10 + 1 = 31. 31 * 10 + 4 = 314.

A 314

Each pass shifts left one place and adds the next digit.

B 8

That is the plain sum. The times 10 changes it.

C 413

The reversing pattern. Here digits are added in list order.

D 3140

One shift too many. The last digit is added, not shifted.

THE SIGNAL

result * 10 + digit builds a number left to right, in the order the digits arrive.

What is the output of this program?

```
1  temps = [-4, -9, -2, -7]
2  highest = 0
3  for t in temps:
4      if t > highest:
5          highest = t
6  print(highest)
```

A -2

B -9

C 0

D -4

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  temps = [-4, -9, -2, -7]
2  highest = 0
3  for t in temps:
4      if t > highest:
5          highest = t
6  print(highest)
```

No temperature is greater than 0, so highest is never replaced.

A `-2`

The right answer to the question the code was meant to ask. The start of 0 prevents it.

B `-9`

That is the smallest, and the test uses >.

C `0`

Every value is below the starting 0, so it is never replaced.

D `-4`

Only if highest started at temps[0] and nothing beat it. -2 would.

THE SIGNAL

If the answer is a value that is not in the data, the starting value leaked through.

What is the output of this program?

```
1  n = 1203
2  count = 0
3  while n > 0:
4      if n % 10 == 0:
5          count += 1
6      n //= 10
7  print(count)
```

A 0

B 1

C 2

D 4

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  n = 1203
2  count = 0
3  while n > 0:
4      if n % 10 == 0:
5          count += 1
6      n //= 10
7  print(count)
```

Last digits in turn: 3, 0, 2, 1. One of them is 0.

A 0

Stops too early. The 0 is the second digit examined.

B 1

Counts zero digits: 3, 0, 2, 1 contains one zero.

C 2

Counts the final n == 0 too. The loop ends before testing it.

D 4

Counts every digit, ignoring the if.

THE SIGNAL

The loop sees digits from right to left, and stops as soon as n reaches 0.

What is the output of this program?

```
1  n = 9
2  for d in range(2, n):
3      if n % d == 0:
4          print("not prime", d)
5          break
6  else:
7      print("prime")
```

A prime

B not prime 9

C not prime 2

D not prime 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  n = 9
2  for d in range(2, n):
3      if n % d == 0:
4          print("not prime", d)
5          break
6  else:
7      print("prime")
```

d = 2: 9 % 2 is 1. d = 3: 9 % 3 is 0, so print and break.

A `prime`
9 has a divisor, 3, so break fires and the else is skipped.

B `not prime 9`
range(2, 9) stops at 8. 9 is never tried.

C `not prime 2`
9 % 2 is 1, not 0. 9 is odd.

D `not prime 3`
3 is the first divisor found, then break stops the search.

THE SIGNAL

The search reports the first divisor it finds, in the order range produces them.

What is the output of this program?

```
1  data = [5, 1, 4, 2]
2  for i in range(len(data) - 1):
3      if data[i] > data[i + 1]:
4          data[i], data[i + 1] = data[i + 1], data[i]
5  print(data)
```

A [1, 4, 2, 5]

B [1, 2, 4, 5]

C [1, 5, 4, 2]

D [5, 4, 2, 1]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  data = [5, 1, 4, 2]
2  for i in range(len(data) - 1):
3      if data[i] > data[i + 1]:
4          data[i], data[i + 1] = data[i + 1], data[i]
5  print(data)
```

i = 0: [1, 5, 4, 2]. i = 1: [1, 4, 5, 2]. i = 2: [1, 4, 2, 5].

A `[1, 4, 2, 5]`

One pass carries 5 to the end; the rest is not yet sorted.

B `[1, 2, 4, 5]`

Fully sorted needs another pass. There is no outer loop.

C `[1, 5, 4, 2]`

Stops after the first comparison. The loop makes three.

D `[5, 4, 2, 1]`

That is descending order. The test swaps when the left is bigger.

THE SIGNAL

One pass of bubble sort only guarantees the largest value reaches the end.

What is the output of this program?

```
1  words = ["a", "bb", "ccc", "dd"]
2  total = 0
3  for w in words:
4      if len(w) == 2:
5          total += len(w)
6  print(total)
```

A 2

B 4

C 8

D 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  words = ["a", "bb", "ccc", "dd"]
2  total = 0
3  for w in words:
4      if len(w) == 2:
5          total += len(w)
6  print(total)
```

"bb" and "dd" qualify, and each adds its length, 2.

A 2

Counts the matching words. The body adds their lengths.

B 4

Two words of length 2 each add 2.

C 8

Adds every length, ignoring the condition.

D 3

Picks up "ccc". Its length is 3, not 2.

THE SIGNAL

Read what is added: += 1 counts, += value sums.

What is the output of this program?

```
target = 7
for n in [2, 7, 7, 3]:
    if n == target:
        print("found", end=" ")
        break
else:
    print("missing", end=" ")
print("done")
```

A. found found done

B. found missing done

C. found done

D. missing done

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  target = 7
2  for n in [2, 7, 7, 3]:
3      if n == target:
4          print("found", end=" ")
5          break
6  else:
7      print("missing", end=" ")
8  print("done")
```

The first 7 triggers the break. The second 7 is never examined.

A `found found done`
break stops the loop at the first 7.

B `found missing done`
break skips the else.

C `found done`
First match, break, else skipped, then the line after the loop.

D `missing done`
7 is in the list, so break fires.

THE SIGNAL

A search with break finds the first match only, and the else runs only when there was no match.

What is the output of this program?

```
1  n = 120
2  rev = 0
3  while n > 0:
4      rev = rev * 10 + n % 10
5      n //= 10
6  print(rev)
```

A 021

B 12

C 210

D 21

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  n = 120
2  rev = 0
3  while n > 0:
4      rev = rev * 10 + n % 10
5      n //= 10
6  print(rev)
```

rev: 0, then 2, then 21. n: 12, then 1, then 0.

A `021`

*An int never prints a leading zero. 0 * 10 + 0 is just 0.*

B `12`

Drops the 0 but does not reverse the rest.

C `210`

The trailing 0 of 120 becomes a leading 0, which vanishes.

D **`21`**

The first digit taken is 0, so rev stays 0; then 2, then 21.

THE SIGNAL

Trailing zeros vanish when a number is reversed as an int.